ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

# Nhập môn An toàn thông tin

## Chương 5. **Tạo lập và quản lý khóa**

# Các nội dung trình bày

- Trao chuyển khóa
  - Khóa phiên vs. Khóa trao chuyển
  - Các phương pháp tạo lập sử dụng mật mã đối xứng và CK
  - Cơ chế sinh khóa
- Hạ tầng chứng chỉ khóa CK
  - Khái niệm chứng chỉ và quản lý
- Làm quen với giao thức bảo mật

# Cú pháp

- $X \rightarrow Y : \{ Z \parallel W \} k_{X,Y}$
  - $X$ gửi tới $Y$ một thông điệp tạo bởi 2 phần nội dụng $Z$ và $W$ được mã hóa bởi khóa đối xứng $k_{X,Y}$, chia sẻ giữa hai bên $X$ & $Y$
- $A \rightarrow T : \{ Z \} k_A \parallel \{ W \} k_{A,T}$
  - $A$ gửi $T$ một thông điệp mật mã bao gồm 2 đoạn mã: 1) nội dung $Z$ mã bởi khóa riêng $k_A$ *của* $A$, và 2) nội dung $W$ mã hóa bởi khóa đối xứng $k_{A,T}$, chia sẻ giữa $A$ & $T$
- $r_1$, $r_2$ các giá trị số được sinh ngẫu nhiên, không lặp lại

# Khoa phiên – Khóa chuyển

- Alice muốn gửi thông điệp *m* tới Bob
  - Giả định có thể dùng hệ khóa công khai (CK)
  - Alice đã có khóa CK của Bob: $Z_B$
- Giao thức được đề xuất
  - Alice sinh ngẫu nhiên khóa $k_s$ và dùng nó mã hóa *m*
    - Khóa này chỉ dùng cho việc chuyển thông điệp này
    - Được gọi là *khóa phiên*
  - Cô ta mã hóa $k_s$ bằng khóa CK của Bob $Z_B$
    - $Z_B$ sẽ được sử dụng để mã hóa và trao chuyển mọi khóa phiên
    - Do đó được gọi là *khóa trao chuyển* (hay khóa chuyển)
  - Alice gửi Bob: $\{ m \} k_s \,||\, \{ k_s \} Z_B$

# Tại sao lại dùng *khóa phiên*?

- Nhằm hạn chế lượng thông tin giao thông cùng được mã hóa bởi 1 khóa duy nhất
  - Một kỹ thuật cơ bản nhằm hạn chế cơ hội của kẻ tấn công phá mã
- Ngoài ra có thể giúp khắc phục một số tình huống
  - Ví dụ : Các thông điệp giữa A và B là rất ngắn gọn và lặp lại, kiểu như "BUY" hay "SELL". Khi này Eve có thể dễ dàng xác định được các mã { "BUY" } $Z_B$ and { "SELL" } $Z_B$, tức là Eve có thể đoán được các nội dung đơn giản này khi nghe trộm.

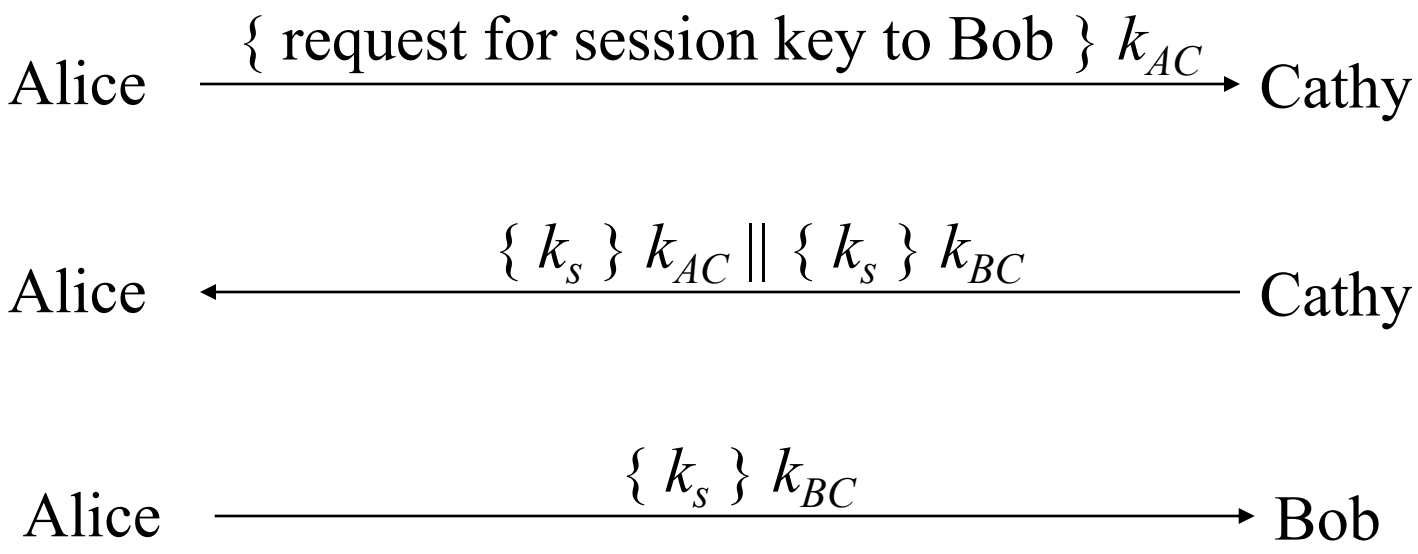
# Các thuật toán trao chuyển khóa

- Mục tiêu: A & B có thể tạo ra khóa phiên đối xứng, chia sẻ giữa 2 bên (mà ko sử dụng khóa chuyển)
  - Lưu ý là khóa phiên ko thể được chuyển như 1 bản rõ
    - Kẻ xấu có thể nghe trộm đường truyền
    - Khóa phải được mã hóa hoặc được tạo ra từ dữ liệu được chia sẻ giữa 2 bên mà kẻ nghe trộm không tạo ra được
  - Alice, Bob có thể sử dụng tác nhân người thứ 3 trung gian tin cậy
  - Giả thiết tất cả các giao thức, thuật toán là công khai, ko che giấu
    - Dữ liệu mật chỉ bao gôm các khóa, thông tin hỗ trợ cho việc tạo khóa mà chỉ A& B được biết
    - Bất cứ dữ liệu giao thông nào (giữa A&B) phải coi là có thể bị nghe trộm

# Trao chuyển khóa đối xứng

- Vấn đề khởi đầu: Alice, Bob cần bắt đầu ra sao?
  - Không thể gửi khóa như bản rõ!
  - Không có cách nào để mã hóa
- Hãy giả thiết có 1 người trung gian tin cậy: Cathy
  - Alice & Cathy chia sẻ bí mật khóa đối xứng $k_{AC}$
  - Bob & Cathy chia sẻ bí mật khóa đối xứng $k_{BC}$
  - Cần tìm cách để tạo ra khóa phiên giữa A&B: $k_s = k_{AB}$

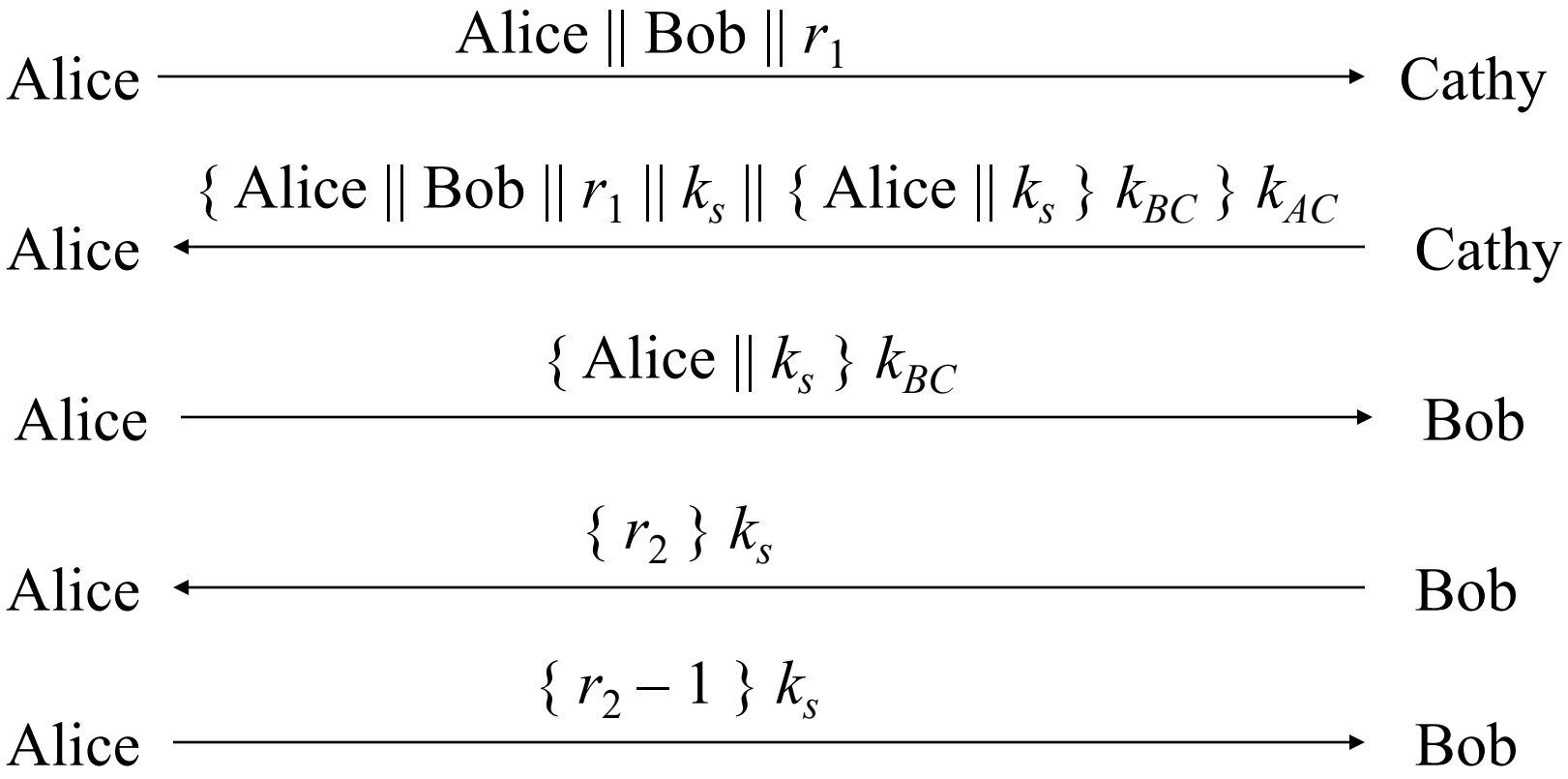
# Giao thức đơn giản



# Điểm yếu

- Làm sao Bob biết được thực sự đang nc với Alice?
  - Tấn công phát lại (Replay attack) : Eve “ghi chép” lại mọi thông tin giao thông giữa  Alice và Bob, rồi sau đó lựa lúc thích hợp để phát lại -- Bob có thể cho là mình đang nc với Alice trong khi thực ra là nc với Eve
- Phải có cơ chế xác thực danh tính (A & B xác thực lẫn nhau được) và khắc phục được tấn công phát lại

# Giao thức Needham-Schroeder



# Phân tích: Alice talking to Bob

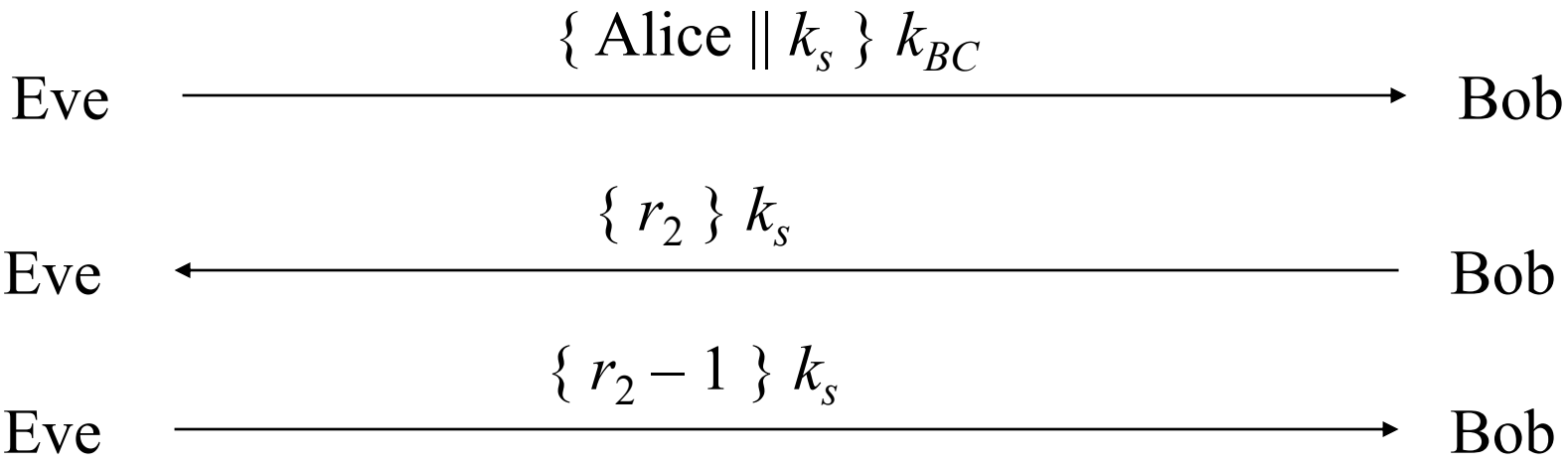
- Với thông điệp thứ 2
  - Được mã hóa bởi khóa mà chỉ A & C có
    - Cho nên A có thể xác định là chỉ có C là đã mã hóa nó
  - Nó đáp ứng khớp với thông điệp 1
    - Vì $r_1$ xuất hiện trong nó là khớp với $r_1$ trong thông điệp 1
- Với thông điệp 3
  - Alice biết rằng chỉ Bob có thể mở đọc nó
    - Tức là chỉ Bob có thể lấy sử dụng được khóa phiên ở trong
  - Do đó bất kỳ thông điệp nào nhận được mà mã hóa bởi khóa phiên là đều đi từ Bob

# Argument: Bob talking to Alice

- Thông điệp 3
  - Quan sát rằng: Mã hóa bởi khóa $K_{BC}$ chỉ B và C biết
    - Do đó Bob biết là thông điệp được tạo bởi C
  - Khi giải mã B thấy tên Alice và khóa phiên $k_s$ , nên Bob kết luận Cathy đã tạo ra k, còn Alice là đối tượng xin kết nối
- Thông điệp 4 & 5
  - Cơ chế thách thức – đáp ứng để B xác thực đối tượng là Alice thực sự chứ không phải kẻ mạo danh
    - Chỉ mình Alice có $k_s$ nên đáp ứng chính xác trong bước 5
    - Trong tấn công phát lại, Eve không thể lấy được $r_2$ từ thông điệp 4 nên ko thẻ đáp ứng chính xác trong t. điệp 5

# Vấn đề Denning-Sacco

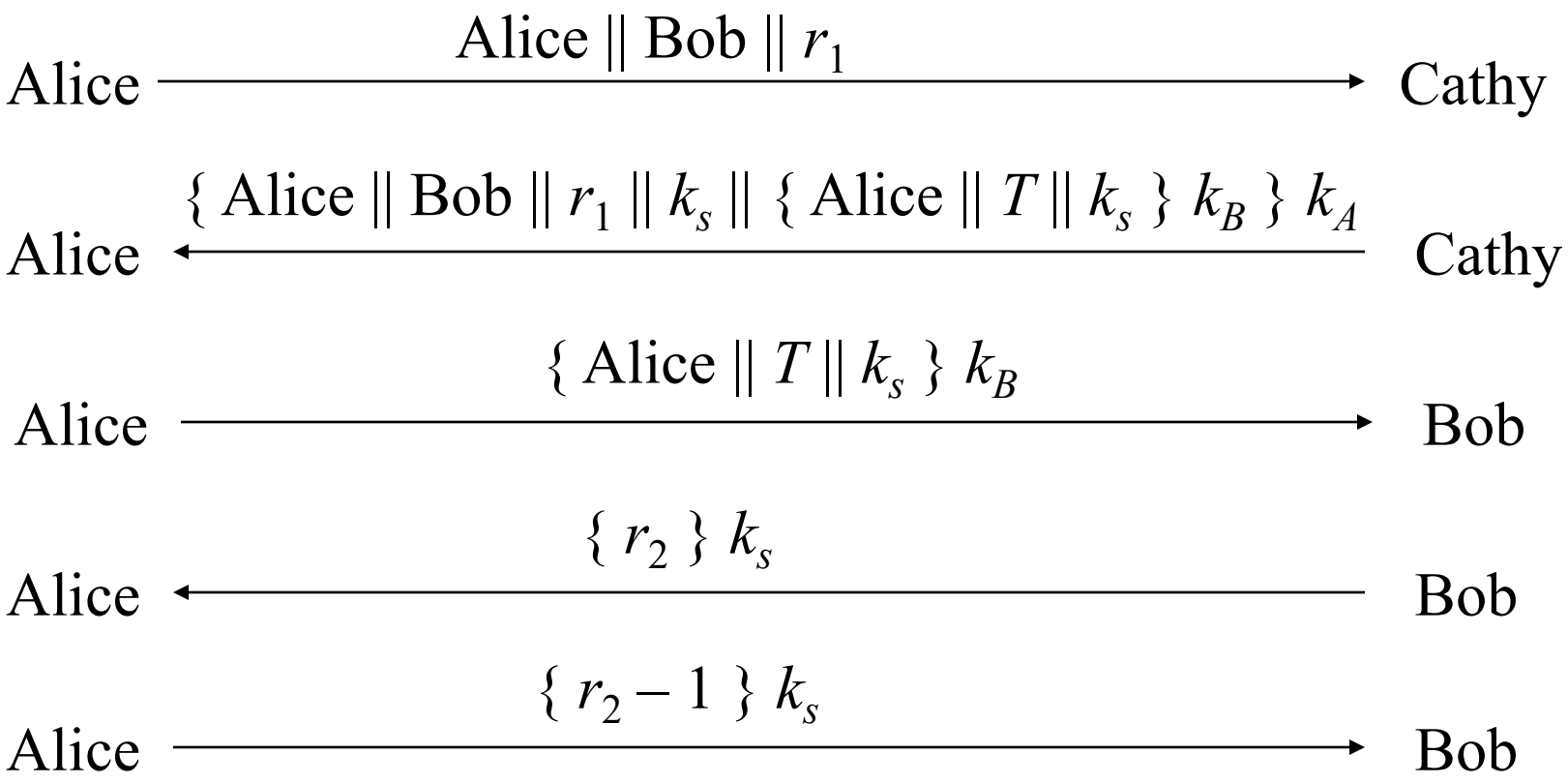
- Ta đang giả thiết mọi khóa là bí mật
- Câu hỏi: giả sủ Eve tình cờ may mắn tóm được 1 khóa phiên cũ. Liệu Eve có khai thác được nó?
  - Câu trả lời là Eve hoàn toàn có thể khai thác thành công, để giả mạo Alice như dưới đây



# Tấn công khóa phiên cũ

- Trong tấn công này Eve sẽ giả mạo Alice mà Bob không thể phát hiện
- Điểm yếu của giao thức NS: Eve có thể phát lại thông điệp số 3 mà Bob không thể nhận ra vì Eve có ks nên có thể đáp ứng được trong th điệp 5
  - Chú ý ở sơ đồ trên nó là bước 1
- Giải pháp khắc phục: có nhiều cách
  - Cách phổ biến nhất là sử dụng nhãn thời gian *T* to để phát hiện tấn công phát lại

# Giao thức Needham-Schroeder với điều chỉnh Denning-Sacco



# Vẫn chưa hết điểm yếu …

- Nếu đồng hồ giữa các bên không đồng bộ thì có thể dẫn đến từ chối không kết nối hoặc chấp nhận kết nối với kẻ giả mạo phát lại
- Khắc phục: Giao thức Otway-Rees protocol
  - Đọc thêm: Bishop's text

# Kerberos

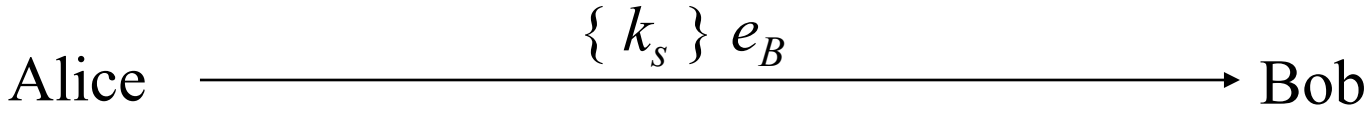
- Một giải pháp hệ thống cung cấp cơ chế xác thực và bảo mật kênh
  - Dựa vào GT Needham-Schroeder với điều chỉnh Denning-Sacco
  - Một máy chủ trung tâm sẽ đóng vai trò bên thứ 3 tin cậy ("Cathy")
- Ticket
  - Vé được máy chủ phát với các thông tin thiết lập khóa phiên
- Ứng dụng: các hệ thống lớn nhiều máy chủ, ví dụ như mạng xã hội FB, hệ đa dịch vụ kiểu Google

# Ý tưởng cơ bản

- User *u* authenticates to Kerberos server
  - Obtains ticket $T_{u,TGS}$ for ticket granting service (TGS)
- User *u* wants to use service *s*:
  - User sends authenticator $A_u$, ticket $T_{u,TGS}$ to TGS asking for ticket for service
  - TGS sends ticket $T_{u,s}$ to user
  - User sends $A_u$, $T_{u,s}$ to server as request to use *s*
- Details in Bishop's text

# Trao chuyển khóa công khai

- Ở đây ta xét sử dụng các khóa chuyển liên quan
  - $e_A$, $e_B$: khóa ck của Alice & Bob được tất cả biết t
  - $d_A$, $d_B$: khóa riêng bí mật của A & B
- Hãy xét giao thức "ngây thơ" được đề xuất đơn giản
  - $k_s$ là khóa phiên cần thiết lập giữa A & B



# Nhược điểm và giải pháp

- Dễ bị tổn thương với tấn công giả mạo hay phát lại
  - Bởi vì $e_B$ là công khai ai cũng có, Bob không thể biết là thông điệp nhận là đến từ Alice
- Cách chữa đơn giản là sử dụng khóa mât riêng của Alice
  - $k_s$ là khóa phiên muốn chuyển

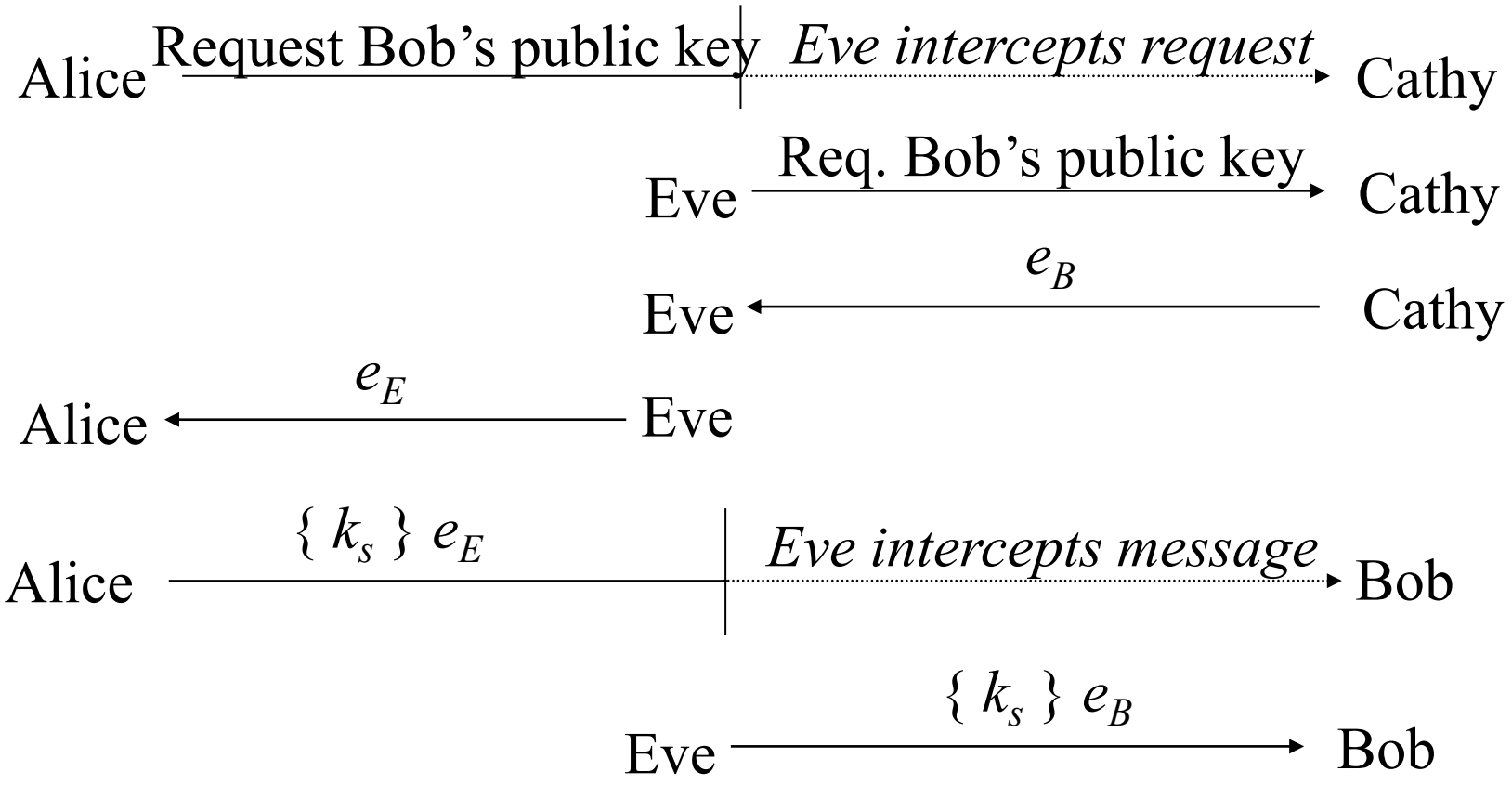
$$\text{Alice} \xrightarrow{\text{Alice} \,||\, \{\, \{\, \text{Alice} || k_s \,\}\, d_A \,\}\, e_B} \text{Bob}$$

  - **Quiz:** Liệu Eve giả mạo được Alice để trao chuyển $k_s$ với Bob để kết nối thành công ?

# Chú ý

- Alice có thể mở rộng, gộp luôn vào một thông điệp mã hóa bởi $k_s$
- Tuy nhiên giao thức này giả thiết A & B đã có khóa CK của nhau
  - Nếu không, mỗi bên sẽ phải tìm lấy KCK của bên kia từ một public server
  - Kẻ tấn công Eve có thể thực hiện tấn công *kẻ-ngồi-giữa* (*man-in-the-middle*) để chiếm khóa phiên (trong slide tới Cathy là public server lưu giữ và cung cấp KCK)
    - Giải pháp cho tấn công này là cơ chế gắn liềnkhóa với danh tính của chủ (binding identity to keys) sẽ trình bày sau

# Man-in-the-Middle Attack



# Vấn đề sinh khóa

- Mục đích: sinh khóa phiên sao cho khó đoán
- Bài toán đặt ra: Từ một tập giá trị khóa tiềm năng cho trước *K*, hãy chọn một phần tử một cách ngẫu nhiên
  - Tương đương với BT lấy ngẫu nhiên 1 số trong đoạn số nguyên 0 .. *K*–1
- Tại sao đây là vấn đề khó: thực sự sinh ngẫu nhiên
  - Thực tế các máy tính chỉ có thể sinh giả-ngẫu nhiên (*pseudo-random*) vì sử dụng các thuật toán tất định
    - Nếu ta cung cấp đầu vào giống hệt nhau cho cùng 2 máy tính giống hệt nhau chạy cùng 1 thuật toán thì kết quả đầu ra giống hệt nhau (hai máy tính chạy song song và có trị số clock như nhau)
    - Hãy so sánh với các phương pháp sinh ngẫu nhiên sử dụng một quá trình vật lý -- vd như tung xúc xắc

# “Random” là gì?

- *Sequence of cryptographically random numbers*: a sequence of numbers $n_1$, $n_2$, … such that for any integer $k > 0$, an observer cannot predict $n_k$ even if all of $n_1$, …, $n_{k-1}$ are known
  - Cách lý tưởng để sinh NN là sử dụng các quá trình vật lý đã được biết
    - Xung ngẫu nhiên
    - Các hiện tượng điện từ trường
    - Các đặt điểm của môi trường tính toán vd như độ trễ trong các thao tác đọc ghi đĩa
    - Ambient background noise

# “Pseudorandom”?

- *Sequence of cryptographically pseudorandom numbers*: sequence of numbers intended to simulate a sequence of cryptographically random numbers but generated by an algorithm
  - Rất khó để thực hiện thông qua các chương trình máy tính
    - Các bộ sinh dựa trên đồng dư tuyến tính [$n_k = (an_{k-1} + b) \bmod n$] → đã bị phá
    - Đồng dư đa thức [$n_k = (a_j n_{k-1}^j + \ldots + a_1 n_{k-1}\, a_0) \bmod n$] → cũng bị phá
    - “phá” nghĩa là: phần tử tiếp theo trong dãy có thể xác định được

# Một số bộ sinh giả NN tốt nhất

- *Strong mixing function*: Các hàm mà các input bit phụ thuộc theo quan hệ phi tuyến vào tất cả các output bit
  - Ví dụ: DES, MD5, SHA-1
  - Các hệ điều hành phỏng UNIX có thể trộn như vd:

    ```
    (date; ps gaux) | md5
    ```

    trong đó "ps gaux" trả lại danh sách thông tin trạng thái của tất cả các tiến trình đang chạy trong hệ thống (tài nguyên bộ nhớ sử dụng …)

# Hạ tầng khóa mật

- Cryptographic Key Infrastructure
- Mục đích: đạt được sự gắn chặt khóa với danh tính chủ thể
- Trong các hệ truyền thống (đối xứng): điều này là không thể vì tất cả các khóa là chia sẻ
  - Thông qua các giao thức tạo lập & trao chuyển khóa
- Với hệ KCK: có thể tạo ra cơ chế “bind identity to public key”
  - Qua đó người ta có thể kết nối an toàn nhờ khóa với danh tính chủ thể đã được gắn xác thực (authentic binding)
  - Lỗi trong cơ chế binding này sẽ dẫn đến sự mất an toàn bảo mật trong liên lạc giữa 2 bên

# Chứng chỉ (Certificates)

- Tạo ra một vật mang thông tin -- token -- bao gồm các trường
  - Danh tính chủ thể (ví dụ, Alice)
  - KCK tương ứng
  - Nhãn thời gian (thời điểm phát hành chứng chỉ)
  - Một số thông tin khác (vd. Danh tính bên ký, phát hành)

  được ký bởi một nhà phát hành thẩm quyền (Cathy)

$$C_A = \{ e_A \,||\, \text{Alice} \,||\, T \}\, d_C$$

# Khai thác

- Bob tìm lấy chứng chỉ của Alice, phát hành và ký bởi Cathy
  - Nếu B đã biết KCK của Cathy thì Bob có thể giải mã và lấy thông tin trong chứng chỉ và kiểm chứng
    - CC được phát hành khi nào?
    - Có thật chủ thể là Alice?
  - Giờ B có KCK của A
- Tồn tại: Bob cần phải có KCK của Cathy để kiểm tra chứng chỉ
  - Thực tế các cơ quan CA chỉ quản lý một vùng địa phương hẹp
  - Điều này đẩy giải pháp lên một "cấp" trên
  - Có 2 tiếp cận: Merkle's tree, chuối chư kỹ (signature chains)

# Certificate Signature Chains

Các thủ tục cần tiến hành

- Lập chứng chỉ
  - Tạo giá trị băm của chứng chỉ
  - Mã hóa giá trị băm này với chữ ký mật của CA
- Xác minh
  - Bob cần có KCK của cơ quan phát hành -- CA
  - Giải mã lấy giá trị băm nói trên
  - Tính toán giá trị băm độc lập (theo tt của CC) và so sánh
- Tồn tại: Làm sao Bob có được KCK của CA?

# X.509 Chains

- Một số thành phần của CC theo chuẩn X.509v3:
  - Phiên bản
  - Số seri
  - Định danh thuật toán sinh chữ ký: thuật toán băm
  - Tên nhà phát hành; định danh duy nhất
  - Thời hạn của CC
  - Tên chủ thể KCK; định danh duy nhất
  - KCK được phát hành (cho chủ thể này)
  - Chữ ký của CA: giá trị băm được ký bởi thuật toán ký nói trên

# Các bước xác minh CC X.509

- Tim lấy  KCK của cơ quan CA
- Giải mã chữ ký
  - Lấy được giá trị băm – hash value -- của thông tin CC (hash of certificate)
- Tính lại giá trị băm từ thông tin trong và so sánh
  - Nếu so sánh thấy khác tức là CC này có vấn đề, loại!
- Kiểm tra thời hạn CC
  - CC còn trong hạn sử dụng?

# Các cơ quan phát hành -- Issuers

- *Certification Authority (CA)*: cơ quan có thẩm quyền phát hành CC
  - Trong thực tế cần nhiều CA để cấp phát & quản lý CC cho nhiều vùng khác nhau → làm thế nào B có thể xác minh được CC của A khi không ở cùng 1 vùng (tức là ko có KCK của CA phát hành CC cho A)
  - Alice's CA = Cathy; Bob's CA = Don; làm sao Alice xác minh được CC của Bob và ngược lại?
  - Giải pháp: thêm bước khởi đầu là Cathy & Don cần phát hành chứng chỉ cho nhau (cross-certify)
    - Each issues certificate for the other

# Cơ chế phát hành chéo và ứng dụng cho việc tìm lấy KCK ở xa

- Hệ thống sẽ có các CC :
  - Cathy<<Alice>>: CC của A do C phát hành và ký
  - Dan<<Bob>: CC của B do D phát hành và ký
  - Cathy<<Dan>>: CC của D do C phát hành và ký
  - Dan<<Cathy>>: CC của C do D phát hành và ký
- Các bước để A xác minh CC của B:
  - Alice tìm nhận Cathy<<Dan>>
  - Alice dùng (đã biết) KCK của Cathy để xác thực Cathy<<Dan>>
  - Alice dùng Cathy<<Dan>> để xác thực Dan<<Bob>> và lấy được KCK của Bob

ĐẠI HỌC BÁCH KHOA

25 YEARS ANNIVERSARY SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

# Thank you for your attentions!




soict.hust.edu.vn/ fb.com/groups/soict